DRUNIX HACKATHON 2026 · NPCI × Citi · Track: Real-Time Payments

Revenant

Recover the failed payments worth recovering — and prove every automated decision on DRUNIX.

An autonomous revenue-recovery control plane for payments, with a tamper-evident record of every money decision anchored on NPCI’s permissioned ledger.

Team qbit github.com/adityacosmos24/mini-revenant

THE PROBLEM

Payments fail at national scale — and nobody prices the failures

24.07 bn

UPI transactions in September 2026

worth ₹29.37 lakh crore — in one month, on one rail

7–9%

of UPI transactions failed

~82% business declines (funds, PIN, limits), the rest technical

≈1.8 bn

failed attempts a month

≈₹2.2 lakh crore of attempted value at today's volume and a 7% rate

Some of these come back on their own. Many could be recovered if asked the right way. Today nobody can tell which — so merchants either chase everything or nothing.

Sources: NPCI UPI statistics, Sept 2026 · industry analysis of NPCI decline data, FY23 · failed-attempt value estimated at the average ticket (₹1,220)

WHY IT STAYS UNSOLVED

Merchants see one number, so they make blind moves

THE AGGREGATE HIDES THE INCIDENT

An 8-hour collapse in international card acceptance:

8.8% → 29.6%

failure rate in the affected slice — while the overall rate moved by only a few points, the wobble every dashboard ignores. (Our 75,000-payment simulation.)

Blind move 1 — retry everything

Pays for attempts that were never coming back, annoys customers, adds load on issuers and the network — and risks charging someone twice.

Blind move 2 — switch processors

A multi-month migration decided on a hunch, because nobody can prove whose fault the failures were.

THE NEW PROBLEM

Software now moves the money. Who can prove it followed the rules?

Recovery is becoming automated and AI-assisted. When a merchant, PSP, bank or customer disputes an automated action, the only evidence is the operator’s own database — which the operator can edit.

There is no shared, tamper-evident record. That is what DRUNIX is for.

THE SOLUTION

One closed loop over every payment

1 · Detect
Failure rate per slice vs its own history; five gates

2 · Diagnose
Root cause by excess failures: the slice that changed

3 · Quantify
Every failed payment priced by a calibrated model

4 · Decide
Five options by expected value, including do nothing

5 · Gate
Twelve policy rules, all checked; any DENY wins

6 · Act
Idempotency key first: never a double charge

7 · Verify
Direct, assisted or organic; organic earns zero

8 · Learn
Promised vs delivered, published live

+ ANCHOR ON DRUNIX Every policy decision is hashed into a Merkle batch; the root is written to DRUNIX — off the payment path.

ONE PAYMENT, START TO FINISH

A ₹4,800 card payment fails 3-D Secure

The engine prices all five options and picks the best (✓)

Option	P(recovery)	Expected value
✓ **Alternate gateway**	62%	**₹3,116**
Alternate method	34%	₹1,696
Payment link	28%	₹1,382
Plain retry	9%	₹452
Do nothing	—	₹0

A retry bot sends the card into the same 3-D Secure wall. Revenant reroutes it — and on a domestic insufficient-funds decline, the same engine picks a payment link.

THEN THE POLICY GATE

12 rules, all checked

Kill switch · captured-payment guard · opt-out · terminal failures · attempt limit · cooldown · daily count and spend · hourly blast radius · positive EV · approval above ₹25,000 · no retry into an incident

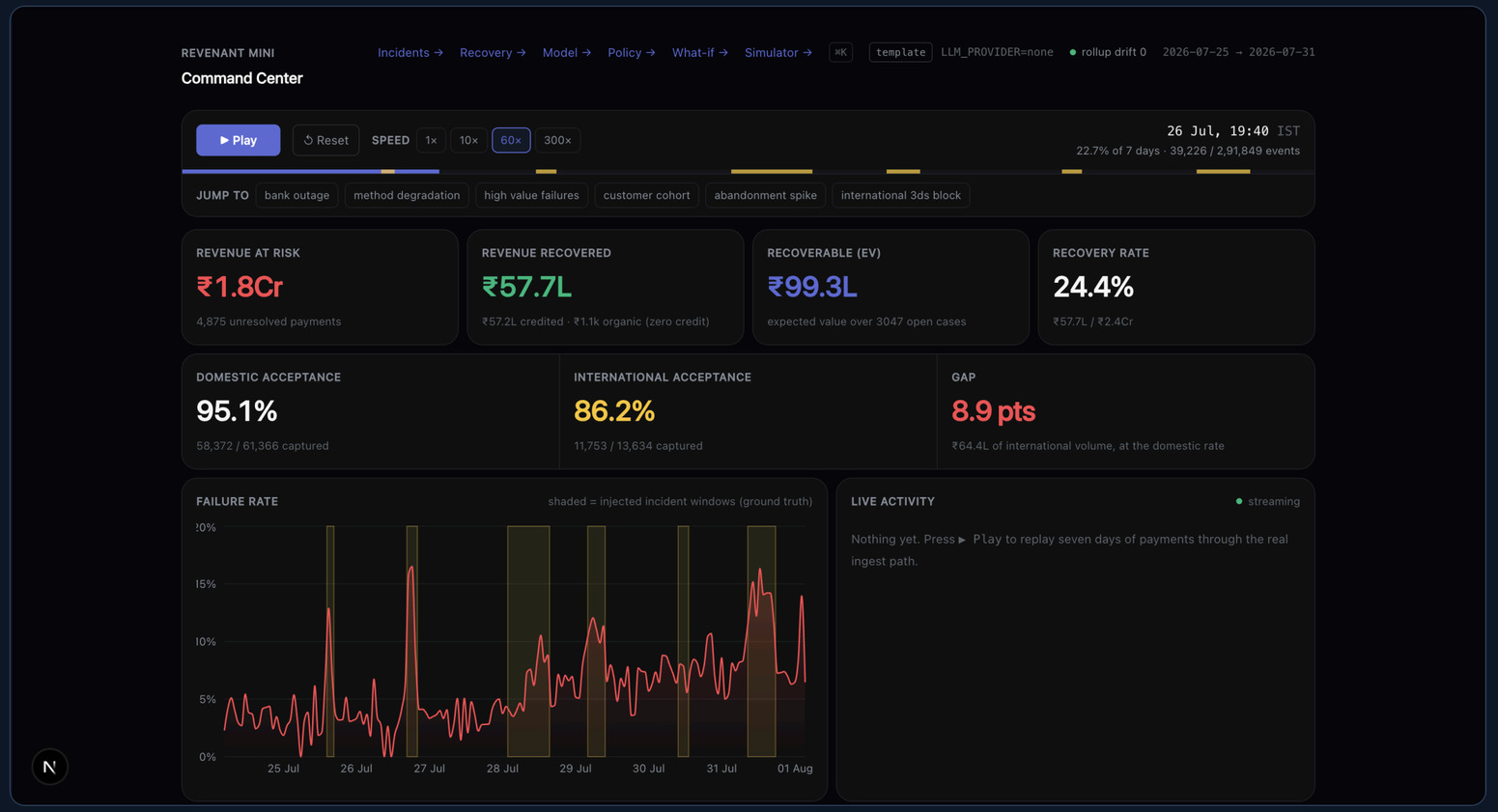
ALLOW → act → anchor

Stored with a hash of its inputs, then anchored on DRUNIX.

Figures computed by Revenant’s strategy engine for this payment (₹4,800, card, CROSS_BORDER family, measured baseline odds).

WORKING PROTOTYPE

Command Center: the money and the gap



Money, with its inputs

At risk, recovered, expected — each printed beside the integers it came from

The acceptance gap

Domestic vs international, side by side

Incidents vs answer key

Injected incident windows shaded on the live failure-rate chart

Eight more pages

Incidents · Recovery · Policy · Model · What-if · Simulator · Audit

HOW DRUNIX IS USED

Every money decision, provable on DRUNIX

Policy decision

Every ALLOW, DENY and approval, stored in Postgres with its inputs

Merkle batch

Up to 256 decisions hashed into one 32-byte root

revenant-anchor

Chaincode called through the Fabric Gateway on a Lite Peer

On DRUNIX

Co-endorsed by two organisations, committed by Committing Peers

Verify any decision later: re-hash the row as it stands → fold at most 8 sibling hashes → compare with the root on DRUNIX. Match = verified. No match = the audit page names what changed.

Only roots leave the database

A root, a leaf count and a time range. Never a payment, an amount or a customer.

Never on the payment path

A slow or down ledger delays the evidence, never a decision or a payment.

MULTI-ORGANISATION NETWORK DESIGN

The operator can never write alone

PSP

Runs Revenant and submits anchors

Lite Peers + Committing Peer

Acquiring bank

Co-endorses; checks its merchants' decisions

Lite Peer + Committing Peer

Auditor

Co-endorses; verifies any decision on request

Lite Peer + Committing Peer

Merchant

Optional: verifies its own decisions

Lite Peer, or read-only

ENDORSEMENT POLICY — NO ROOT LANDS WITHOUT A SECOND ORGANISATION

```
AND('PSPMSP.peer', OR('AcquirerMSP.peer', 'AuditorMSP.peer', 'MerchantMSP.peer'))
```

On DRUNIX's two-org test network, Fabric 2.5's default **MAJORITY** endorsement already requires both organisations — the guarantee is demonstrable from day one.

DRUNIX ELEMENTS

What we use, and how far along it is

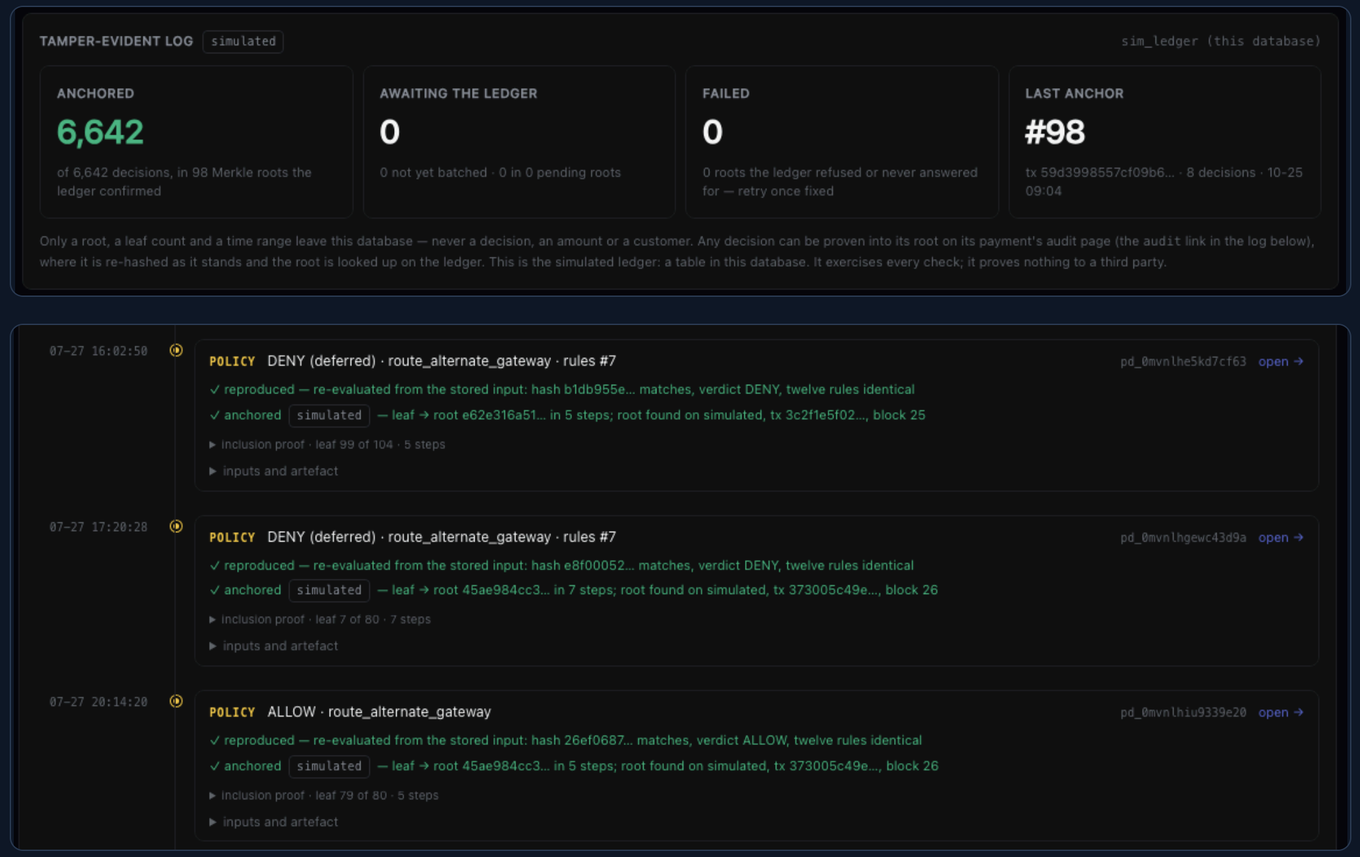
Element	How Revenant uses it	Status
Chaincode (TypeScript)	Anchor + GetAnchor; idempotent by root	**Built**
Fabric Gateway SDK	Stock client signs and submits from the API	**Built**
Lite / Committing Peers	Endorse on Lite Peers next to each org’s apps	**Built (client)**
Multi-org network	PSP + acquirer + auditor co-endorse each anchor	**Designed**
Private data / KeyDB	Leaf hashes shared only with each auditor	**Planned**
YugabyteDB state DB	Auditors query anchors by org and time	**Planned**

NEXT MILESTONE Deploy revenant-anchor on the DRUNIX test network and run the round trip — the steps are in ledger/README.md.

Built = in the repository and tested · Designed = specified, not yet deployed · Planned = on the roadmap

PROOF, FROM A FULL REPLAY

Anchored end to end in a full replay



6,642

policy decisions anchored in 98 Merkle roots — 0 pending, 0 failed

8 of 8

decisions in a batch re-hashed and matched to the root on the ledger

4 of 4

tampering attacks caught in tests: edited row, deleted row, forged leaf, missing root

This replay used the simulated ledger; the DRUNIX adapter is the same interface.

QUANTIFIED IMPACT

Measured against a hidden answer key

Seed-42 simulator: 75,000 payments over 7 days, every failed payment with pre-decided counterfactuals. Both strategies face the same 1,025 held-out failed payments.

+₹8 lakh

more recovered than blind retries on the same failures

2.1×

revenue recovered: ₹15.2–15.4 lakh vs ₹7.2 lakh

6% fewer

interventions: 960 vs 1,025 — and better-chosen ones

6 of 6

injected incidents detected

75–80%

of alerts inside a real incident

0

false alarms in the noise windows

89% → 94%

international acceptance: blind retry vs Revenant

Two full replays (original code and the DRUNIX build) agree within 2%. Synthetic data and a simulated gateway; a live pilot uses an A/B holdout.

HOW IT IS BUILT

Guardrails enforced by code, not review

The gate is a type

An unapproved money action does not compile

Idempotent by constraint

Unique keys, never a read-then-write race

Transactional outbox

Event and message commit together

Timeouts reconciled

Unknown outcomes are looked up, never blindly retried

Integer paise only

Money is never rounded by a float

No unlabelled numbers

Every figure says model or baseline, llm or template

349

unit tests on the pure decision logic

115

integration tests on real PostgreSQL

16

ledger tests: every tamper is caught

TypeScript · Bun · Hono · PostgreSQL 16 · Next.js 15 · Hyperledger Fabric Gateway SDK · fabric-contract-api 2.5

ROADMAP

From prototype to a live DRUNIX channel

NOW

Run on DRUNIX

- Deploy revenant-anchor on the two-org test network
- Round trip: anchor, then verify on /audit
- Both orgs endorse every root

NEXT

Pilot with a PSP

- Gateway adapter for live webhooks
- A/B holdout replaces simulated counterfactuals
- Retrain on real failures

THEN

Open the channel

- Acquirer and auditor join as endorsers
- Private data collections per merchant
- YugabyteDB for auditor queries

WHY REVENANT

A decision engine, not a retry bot — and every decision is provable

Refuses when acting loses

Every failed payment priced; “do nothing” is always on the ballot

Safe by construction

An unapproved money action does not compile; duplicates hit a unique constraint

Provable on DRUNIX

Any organisation verifies any decision in eight hashes, with no customer data on chain

WHAT WE ASK

A DRUNIX sandbox channel to run the two-org round trip, and an introduction to a PSP for a pilot.

Team qbit

github.com/adityacosmos24/mini-revenant